LangGraph: Graphs for Language Model Applications

LangGraph is a library for building language model applications as graphs. An application is defined by a state object, a set of nodes, and a set of edges. Each node is a function that reads the state and returns an update to it. Each edge says which node runs next. The graph runs node by node until it reaches a terminal node.

State is usually a TypedDict, which is a dictionary type where each key has a declared type. Nodes receive the whole state and return only the fields they change; the framework merges those updates into the state. This makes the flow of information explicit: to understand what a node can see, read the state definition, not the node code alone.

Edges come in two kinds. A normal edge always goes from one node to another. A conditional edge calls a routing function that looks at the state and returns the name of the next node. Conditional edges are how a graph branches: for example, a planner node may route to a retriever node when documents are needed and to a direct-answer node when they are not.

Cycles are allowed. A graph may loop back to an earlier node, for example to retry a failed retrieval with a rewritten query or to regenerate an answer that failed verification. A loop must have a termination condition, such as a retry counter stored in state that stops the loop after one retry. Without a cap, a graph can loop forever on a question it cannot answer.

LangGraph also supports persistence through checkpointers. A checkpointer saves the graph state after each step, so a run can be resumed after a failure. Checkpointers also enable human-in-the-loop workflows: a graph can pause before a sensitive step, wait for a human to approve or edit the state, and then continue from the checkpoint. Interrupts are declared on specific nodes, not added as ad-hoc pauses in node code.

Streaming in LangGraph works by running the graph and yielding events as nodes produce them, rather than waiting for the whole run to finish. Token streaming from a language model node is one kind of event; status events from other nodes are another. A client that ignores unknown event types keeps working when new status events are added later.